AUTOMATA DESIGN

Compiler Construction

Abdul Raffay (23i0587)

Ibrahim Azad (23i3049)

Seven Token Types Covered:

#	Token Type	Mandatory?	Regex
1	Integer Literal	Yes	[+-]?[0-9]+
2	Floating-Point Literal	Yes	[+-]?[0-9]+\.[0-9]{1,6}([eE][+-]?[0-9]+)?
3	Identifier	Yes	[A-Z][a-z0-9_]{0,30}
4	Single-line Comment	Yes	##[^\n]*
5	Boolean Literal	Choice	(true|false)
6	Punctuators	Choice	[(){}[\],;:]
7	Inc/Dec Operators	Choice	(\+\+|--)

1. Integer Literal

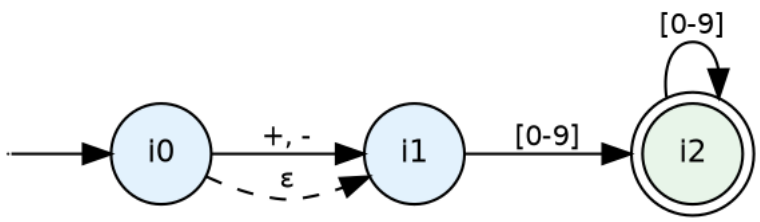
Regular Expression:

```
[+-]?[0-9]+
```

Description: Matches an optional sign (+/-) followed by one or more digits. Valid examples: 42, +100, -567, 0. Invalid: 12.34, 1,000.

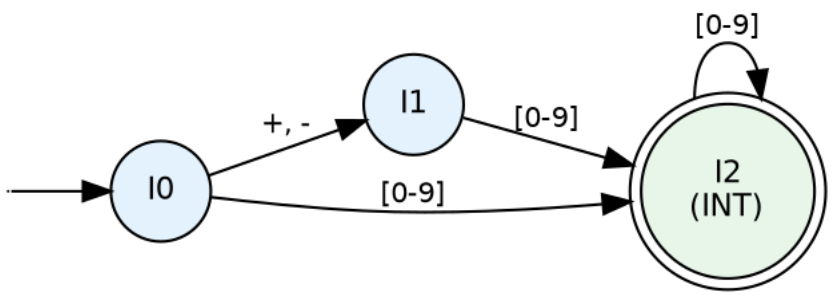
1.1 NFA (with ε-transition for optional sign)

States: i0 (start), i1 (sign processed / skipped via ε), i2 (accepting — digit seen). The ε-transition from i0→i1 models the optional [+-]? part.



1.2 Minimized DFA

After subset construction and minimization: 3 states (already minimal — I0, I1, I2 are all distinguishable by their outgoing transitions).



1.3 Transition Table

State	+, -	[0-9]	Other	Accepting?
I0 (start)	I1	I2	—	No
I1	—	I2	—	No
I2	—	I2	—	Yes (INT)

Note: '—' means transition to dead/trap state (implicit rejection). This DFA has 3 states and is already minimal.

2. Floating-Point Literal

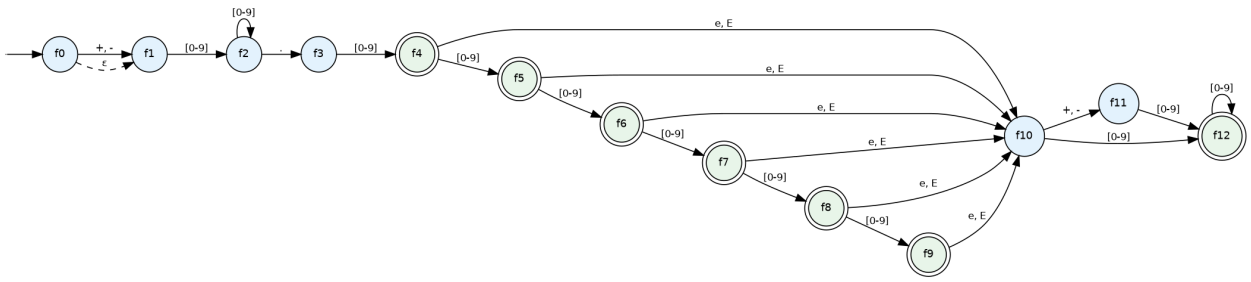
Regular Expression:

```
[+-]?[0-9]+\.[0-9]{1,6}([eE][+-]?[0-9]+)?
```

Description: Optional sign, integer digits, mandatory decimal point, 1-6 decimal digits, optional exponent part (e/E, optional sign, digits). Valid: 3.14, +2.5, -0.123456, 1.5e10, 2.0E-3. Invalid: 3., .14, 1.2345678 (>6 decimals).

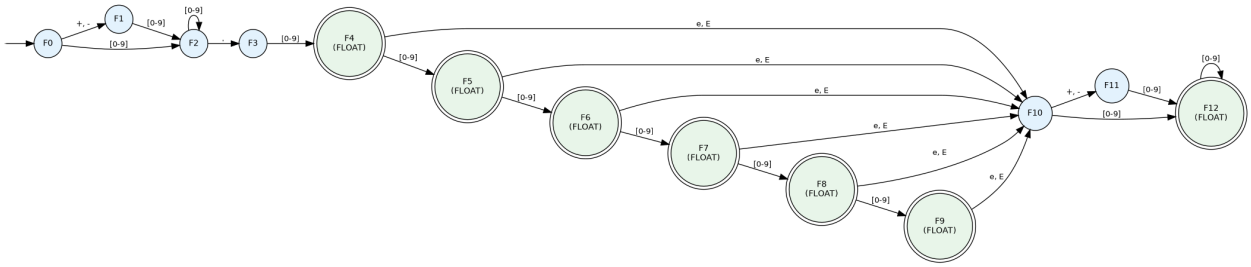
2.1 NFA

States f0-f12. The ε-transitions model optional sign (f0→f1) and optional exponent sign. States f4-f9 are accepting (1-6 decimal digits). State f12 is accepting (exponent digits). The chain f4→f5→...→f9 enforces the {1,6} constraint on decimal places.



2.2 Minimized DFA

13 states total. States F4-F9 enforce 1-6 decimal digits and each is distinguishable (different number of remaining digit transitions). This DFA is already minimal — all accepting states differ in behavior and all non-accepting states differ in reachability.



2.3 Transition Table

State	+, -	[0-9]	.	e, E	Other	Accept?
F0 (start)	F1	F2	—	—	—	No
F1	—	F2	—	—	—	No
F2	—	F2	F3	—	—	No
F3	—	F4	—	—	—	No
F4	—	F5	—	F10	—	Yes (FLOAT)
F5	—	F6	—	F10	—	Yes (FLOAT)
F6	—	F7	—	F10	—	Yes (FLOAT)
F7	—	F8	—	F10	—	Yes (FLOAT)
F8	—	F9	—	F10	—	Yes (FLOAT)
F9	—	—	—	F10	—	Yes (FLOAT)
F10	F11	F12	—	—	—	No
F11	—	F12	—	—	—	No
F12	—	F12	—	—	—	Yes (FLOAT)

13 states total. Already minimal — F4 through F9 are distinguishable because F9 has no digit transition to a next decimal state while F4-F8 do, and the chain propagates distinguishability backward.

3. Identifier

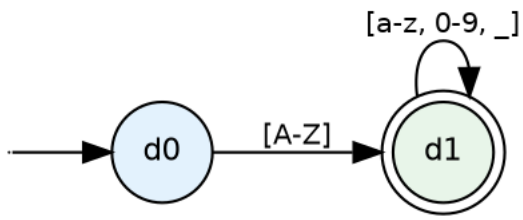
Regular Expression:

```
[A-Z][a-z0-9_]{0,30}
```

Description: Must start with an uppercase letter (A-Z), followed by 0-30 lowercase letters, digits, or underscores. Maximum 31 characters total. Valid: Count, Variable_name, X, Total_sum_2024. Invalid: count, 2Count, myVariable.

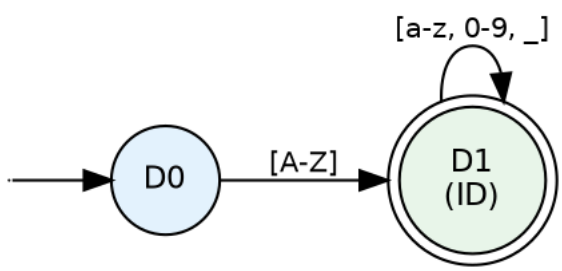
Implementation Note: The {0,30} length constraint is enforced by a counter during scanning (checking total lexeme length ≤ 31) rather than by expanding 31 DFA states. The NFA/DFA below models the core pattern `[A-Z][a-z0-9_]*`.

3.1 NFA



3.2 Minimized DFA

2 states — already minimal. D0 is non-accepting, D1 is accepting. Length constraint (max 31 chars) is checked programmatically.



3.3 Transition Table

State	[A-Z]	[a-z]	[0-9]	_	Other	Accept?
D0 (start)	D1	—	—	—	—	No
D1	—	D1	D1	D1	—	Yes (ID)

2 states, already minimal. Note: [A-Z] from D1 goes to dead state (identifiers cannot contain uppercase after the first character per the regex).

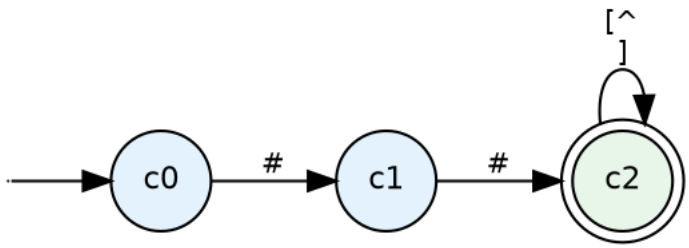
4. Single-line Comment

Regular Expression:

```
##[^\n]*
```

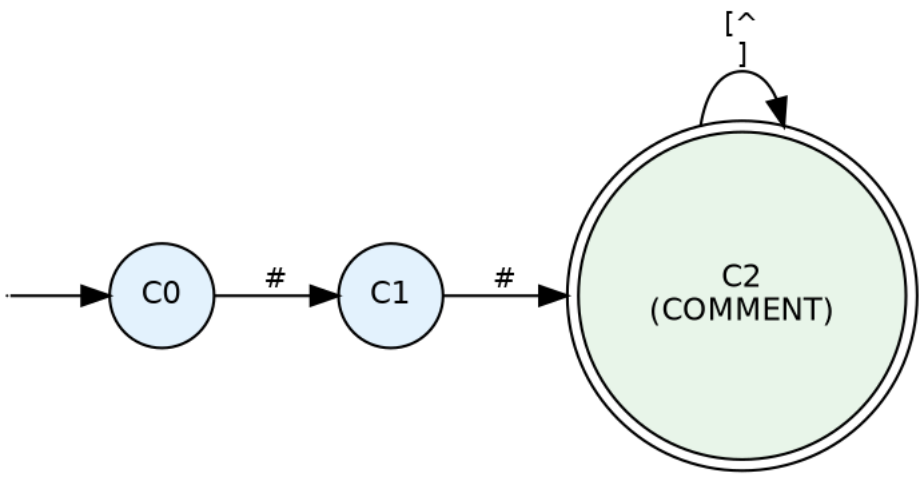
Description: Starts with ## and continues until end of line. The comment body can contain any character except newline. An empty comment ## (with nothing after) is valid.

4.1 NFA



4.2 Minimized DFA

3 states — already minimal.



4.3 Transition Table

State	#	[^\n] (not # or \n)	\n	Accept?
C0 (start)	C1	—	—	No
C1	C2	—	—	No
C2	C2	C2	—	Yes (COMMENT)

3 states, already minimal. C2 accepts any character except \n (which terminates the comment).

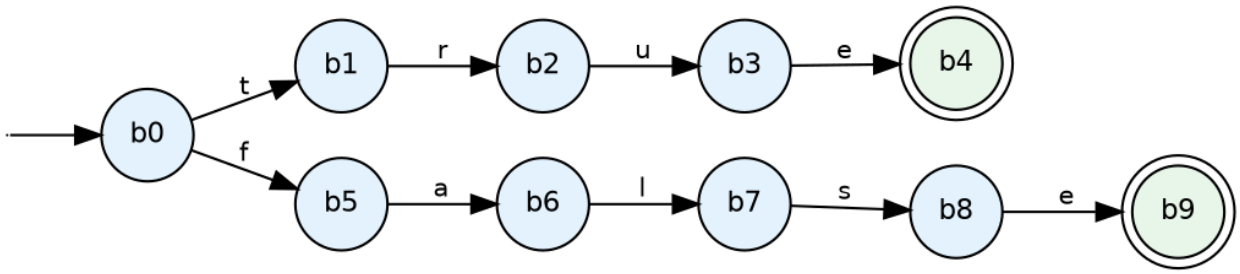
5. Boolean Literal

Regular Expression:

```
(true|false)
```

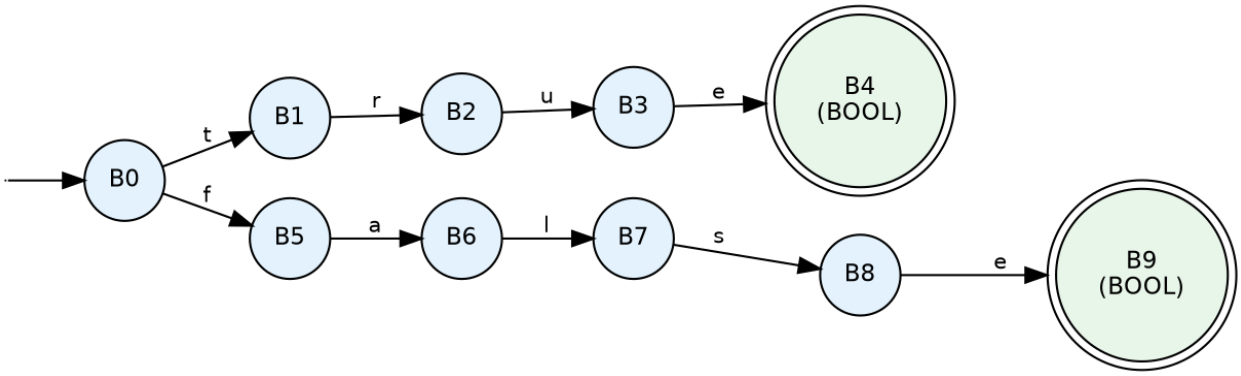
Description: Case-sensitive exact match for 'true' or 'false'. No overlap with identifiers (which must start with uppercase).

5.1 NFA



5.2 Minimized DFA

10 states. The two accepting states B4 ('true') and B9 ('false') are equivalent (both have no outgoing transitions) but are kept separate for clarity. In the combined DFA minimization, they will be merged.



5.3 Transition Table

State	t	r	u	e	f	a	l	s	Accept?
B0	B1	—	—	—	B5	—	—	—	No
B1	—	B2	—	—	—	—	—	—	No
B2	—	—	B3	—	—	—	—	—	No
B3	—	—	—	B4	—	—	—	—	No
B4	—	—	—	—	—	—	—	—	Yes (BOOL)
B5	—	—	—	—	—	B6	—	—	No
B6	—	—	—	—	—	—	B7	—	No
B7	—	—	—	—	—	—	—	B8	No
B8	—	—	—	B9	—	—	—	—	No

B9	—	—	—	—	—	—	—	—	Yes (BOOL)

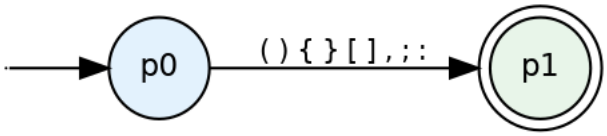
6. Punctuators

Regular Expression:

```
[(){}[\],;:]
```

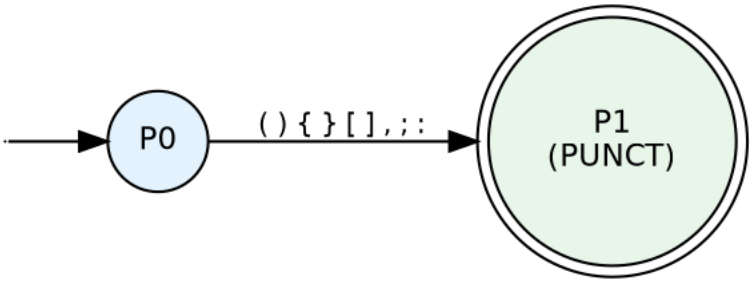
Description: Single-character punctuation marks: `( ) { } [ ] , ; :`

6.1 NFA



6.2 Minimized DFA

2 states — trivially minimal.



6.3 Transition Table

State	`(){}[],;:`	Other	Accept?
P0 (start)	P1	—	No
P1	—	—	Yes (PUNCT)

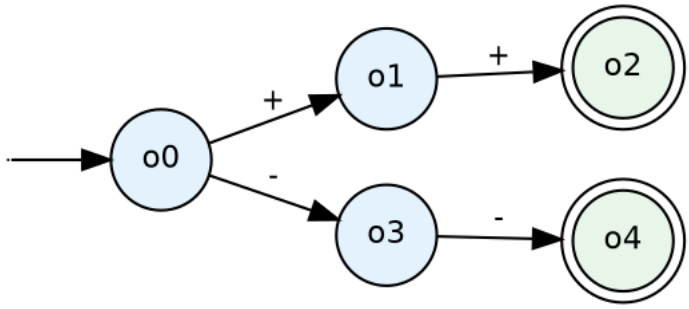
7. Inc/Dec Operators

Regular Expression:

```
(\+\+|--)
```

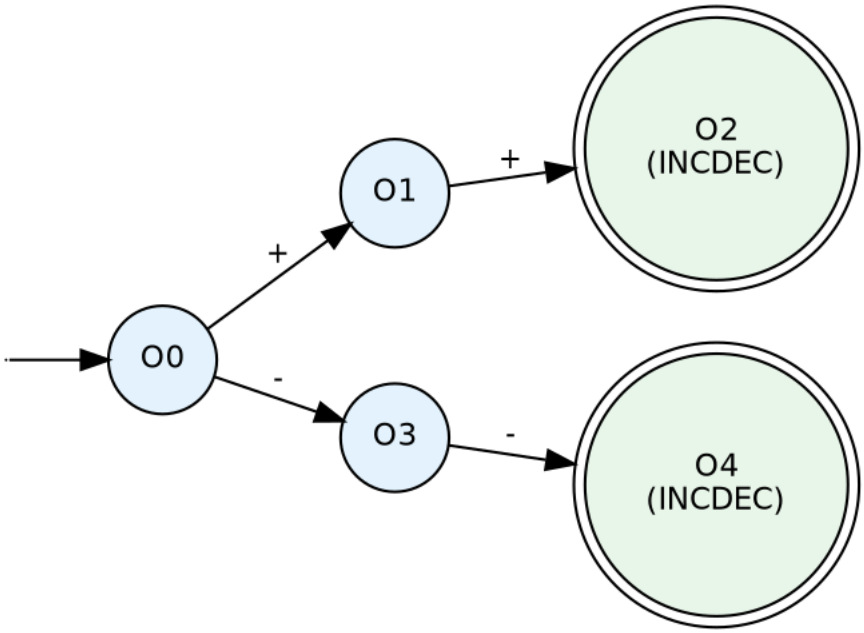
Description: Exactly ++ (increment) or -- (decrement). Two characters each.

7.1 NFA



7.2 Minimized DFA

5 states. O2 and O4 are both accepting with no outgoing transitions — they are equivalent and merged in the combined DFA minimization (→ 4 states). Shown separately here for clarity.



7.3 Transition Table

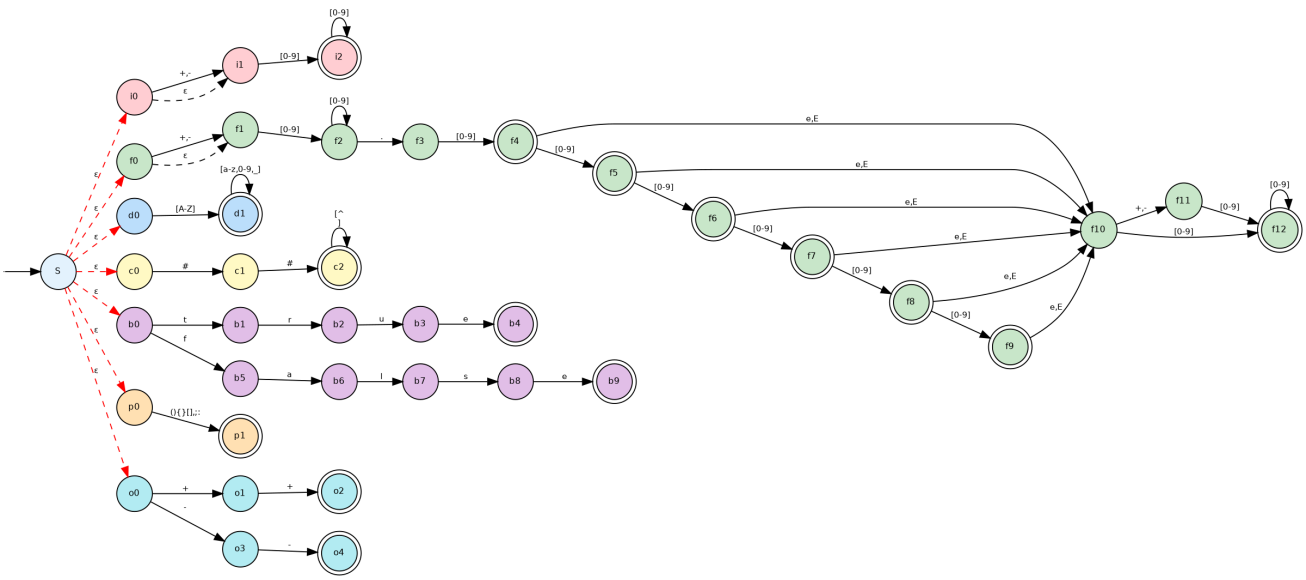
State	+	-	Other	Accept?
O0 (start)	O1	O3	—	No
O1	O2	—	—	No
O2	—	—	—	Yes (INCDEC)
O3	—	O4	—	No
O4	—	—	—	Yes (INCDEC)

8. Combined NFA (All 7 Token Types)

Per the TA clarification, all seven individual NFAs are combined into a single NFA by introducing a new start state S with ε-transitions to each sub-NFA's start state. Each sub-NFA is color-coded for clarity:

Color	Token Type	Sub-NFA States
Red	Integer Literal	i0, i1, i2*
Green	Floating-Point Literal	f0–f12 (f4*–f9*, f12*)
Blue	Identifier	d0, d1*
Yellow	Single-line Comment	c0, c1, c2*
Purple	Boolean Literal	b0–b9 (b4*, b9*)
Orange	Punctuators	p0, p1*
Cyan	Inc/Dec Operators	o0–o4 (o2*, o4*)

The combined NFA has a total of 1 (new start) + 3 (int) + 13 (float) + 2 (id) + 3 (comment) + 10 (bool) + 2 (punct) + 5 (incdec) = **39 NFA states** and 7 ε-transitions from S plus internal ε-transitions for optional parts.



9. Combined DFA via Subset Construction

The combined NFA is converted to a DFA using the subset construction algorithm. Each DFA state represents a set of NFA states. The ε-closures for the optional signs in Integer and Float NFAs mean the start state's ε-closure includes the 'sign-skipped' states.

9.1 Subset Construction Derivation

Start state A = ε-closure({S}) = {i0, **i1**, f0, **f1**, d0, c0, b0, p0, o0}
Note: i1 and f1 are included because i0→i1 and f0→f1 have ε-transitions (modeling the optional sign).

Transitions from each DFA state:

A (start): on + → B{i1,f1,o1}, on `-` → C{i1,f1,o3}, on `[0-9]` → D{i2,f2}*, on `[A-Z]` → E{d1}*, on `t` → F{b1}, on `f` → G{b5}, on # → H{c1}, on `punct` → I{p1}*

B{i1,f1,o1}: on `[0-9]` → D*, on + → JK{o2}*
C{i1,f1,o3}: on `[0-9]` → D*, on `-` → JK{o4}*
D{i2,f2}* [INT]: on `[0-9]` → D, on `.` → L{f3}
E{d1}* [ID]: on `[a-z,0-9,_]` → E
F{b1}: on `r` → M{b2}
G{b5}: on a → N{b6}
H{c1}: on # → O{c2}*
I{p1}* [PUNCT]: no outgoing
JK{o2/o4}* [INCDEC]: no outgoing (J≡K merged)
L{f3}: on `[0-9]` → P{f4}*
M{b2}: on u → Q{b3}
N{b6}: on `l` → R{b7}
O{c2}* [COMMENT]: on `[^\n]` → O
P{f4}* [FLOAT]: on `[0-9]` → S{f5}*, on `e,E` → T{f10,f11}
Q{b3}: on e → UAA{b4}*
R{b7}: on s → V{b8}
S{f5}* [FLOAT]: on `[0-9]` → W{f6}*, on `e,E` → T
T{f10,f11}: on `+,-` → X{f11}, on `[0-9]` → Y{f12}*
Note: ε-closure({f10}) = {f10, f11} because f10 has ε→f11 for optional exponent sign.
UAA{b4/b9}* [BOOL]: no outgoing (U≡AA merged)
V{b8}: on e → UAA*
W{f6}* [FLOAT]: on `[0-9]` → AB{f7}*, on `e,E` → T
X{f11}: on `[0-9]` → Y*
Y{f12}* [FLOAT]: on `[0-9]` → Y
AB{f7}* [FLOAT]: on `[0-9]` → AC{f8}*, on `e,E` → T
AC{f8}* [FLOAT]: on `[0-9]` → AD{f9}*, on `e,E` → T
AD{f9}* [FLOAT]: on `e,E` → T (no more decimal digits allowed)
Z (dead/trap): on any → Z

10. Minimized Combined DFA

10.1 Minimization Analysis

Starting from the 30-state combined DFA (including dead state Z), we apply Hopcroft's DFA minimization algorithm. States are equivalent only if they have the **same token type** AND identical transition behavior.

Initial partition by token type:

Partition	States	Token Type
P1	A, B, C, F, G, H, L, M, N, Q, R, T, V, X, Z	Non-accepting
P2	D	INT
P3	E	ID
P4	I	PUNCT
P5	J, K	INCDEC
P6	O	COMMENT
P7	P, S, W, AB, AC, AD, Y	FLOAT
P8	U, AA	BOOL

Equivalence analysis for mergeable partitions:

P5 {J, K}: Both J and K are accepting (INCDEC) with no outgoing transitions — they transition to Z on all inputs. They are **equivalent** → **merge to JK**.

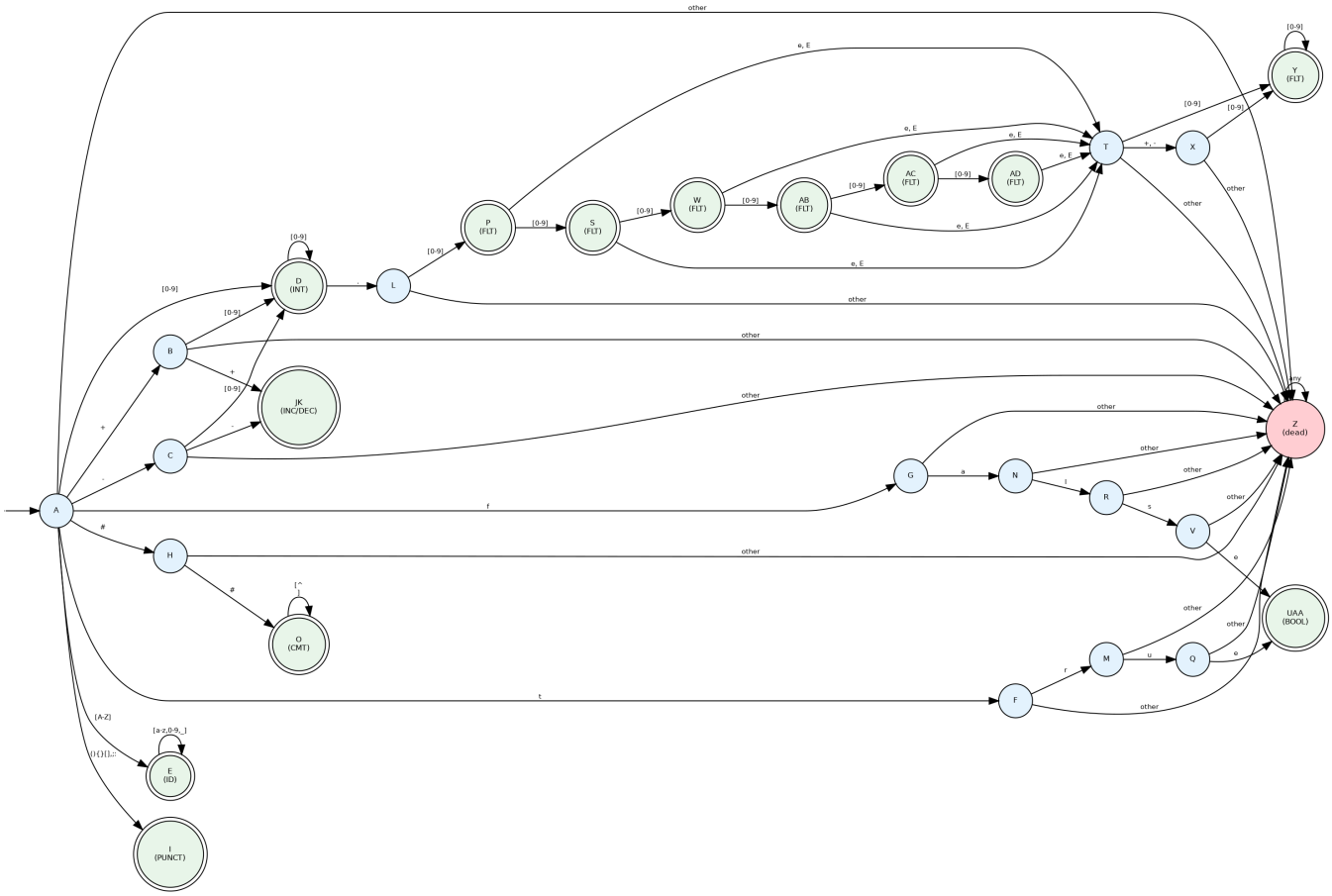
P8 {U, AA}: Both U and AA are accepting (BOOL) with no outgoing transitions — they transition to Z on all inputs. They are **equivalent** → **merge to UAA**.

P7 {P, S, W, AB, AC, AD, Y}: All are FLOAT-accepting but differ in behavior: Y self-loops on [0-9]; AD has no [0-9] successor; AC→AD; AB→AC; W→AB; S→W; P→S. Each state has a unique successor chain. **No merges possible.**

Non-accepting states: Each non-accepting state has unique transitions (e.g., B transitions on +→JK while C transitions on -→JK; F→M on 'r' while G→N on 'a'). **No merges possible.**

Result: 28 states in the minimized combined DFA (30 – 2 merges: J≡K and U≡AA).

10.2 Minimized Combined DFA Diagram


A
B
C
D
(INT)
E
(ID)
F
G
H
I
(PUNCT)
JK
(INC/DEC)
L
M
N
O
(CMT)
P
(FLT)
Q
R
S
(FLT)
T
UAA
(BOOL)
V
W
(FLT)
X
Y
(FLT)
Z
(dead)
AB
(FLT)
AC
(FLT)
AD
(FLT)
other
[0-9]
e, E
+, -
+
-
#
f
t
[A-Z]
[a-z,0-9,_]
{ } ( ) [ ] ;
[^
a
l
s
o
r
u
any

10.3 Combined Minimized DFA — Full Transition Table

Below is the complete transition table for the 28-state minimized combined DFA. Input symbols are grouped by category. '—' denotes transition to dead state Z. Accepting states show their token type.

Part A: States A through O

State	+	-	[0-9]	.	[A-Z]	#	Punct	t	f	r	u	e	a	l	s	Token
A	B	C	D	—	E	H	I	F	G	—	—	—	—	—	—	—
B	JK	—	D	—	—	—	—	—	—	—	—	—	—	—	—	—
C	—	JK	D	—	—	—	—	—	—	—	—	—	—	—	—	—
D*	—	—	D	L	—	—	—	—	—	—	—	—	—	—	—	INT
E*	—	—	E	—	—	—	—	E	E	E	E	E	E	E	E	ID
F	—	—	—	—	—	—	—	—	—	M	—	—	—	—	—	—
G	—	—	—	—	—	—	—	—	—	—	—	—	N	—	—	—
H	—	—	—	—	—	O	—	—	—	—	—	—	—	—	—	—
I*	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	PUNCT
JK*	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	INC/DEC
L	—	—	P	—	—	—	—	—	—	—	—	—	—	—	—	—
M	—	—	—	—	—	—	—	—	—	—	Q	—	—	—	—	—
N	—	—	—	—	—	—	—	—	—	—	—	—	—	R	—	—
O*	O	O	O	O	O	O	O	O	O	O	O	O	O	O	O	CMT

Note for state E (Identifier): transitions on [a-z] characters (r, u, e, a, l, s, and all other lowercase) and [0-9] and _ all loop back to E. For state O (Comment): all characters except \n loop back to O.

Part B: States P through AD, Z

State	+	-	[0-9]	e, E	s	Other	Token
P*	—	—	S	T	—	—	FLOAT
Q	—	—	—	(e)→UAA	—	—	—
R	—	—	—	—	V	—	—
S*	—	—	W	T	—	—	FLOAT
T	X	X	Y	—	—	—	—
UAA*	—	—	—	—	—	—	BOOL
V	—	—	—	(e)→UAA	—	—	—
W*	—	—	AB	T	—	—	FLOAT
X	—	—	Y	—	—	—	—
Y*	—	—	Y	—	—	—	FLOAT
AB*	—	—	AC	T	—	—	FLOAT

AC*	—	—	AD	T	—	—	FLOAT
AD*	—	—	—	T	—	—	FLOAT
Z	Z	Z	Z	Z	Z	Z	(dead)

Note for states Q and V: on input 'e', they transition to UAA (merged accept state for both 'true' and 'false'). State T = ε-closure({f10}) = {f10, f11} handles the optional exponent sign.

Pattern Matching Priority & Disambiguation Strategy

As per Section 3.12 of the assignment, pattern matching priority is **CRITICAL**. The scanner must check patterns in a strict order to resolve ambiguities when multiple token patterns could match the same input. The full priority order is:

Priority	Token Type	Example	In Our 7?
1	Multi-line comments	#* ... *#	No
2	Single-line comments	## comment	Yes
3	Multi-char operators (++, --, ==, !=, etc.)	++, --	Yes (Inc/Dec)
4	Keywords	start, finish, loop	No
5	Boolean literals	true, false	Yes
6	Identifiers	Count, Variable_name	Yes
7	Floating-point literals	3.14, 1.5e10	Yes
8	Integer literals	42, +100, -567	Yes
9	String/character literals	"hello", 'a'	No
10	Single-char operators	+, -, *, /	No
11	Punctuators	( ) { } [ ] , ; :	Yes
12	Whitespace	spaces, tabs, newlines	No

How Priority Is Resolved — Two Mechanisms:

Mechanism 1 — Longest Match (handled by the DFA): The DFA always consumes as many characters as possible. It keeps reading input as long as valid transitions exist, and backtracks to the last accepting state when it gets stuck. For example, seeing ++ the DFA reaches the INCDEC accepting state after both characters rather than stopping at the first +.

Mechanism 2 — Priority Ordering (handled by the scanner code): When two patterns match the *same length* of input, the scanner selects the token type with higher priority. This is implemented in the scanner logic (Task 1.2), NOT in the DFA itself. The DFA's accepting states report which token type(s) matched, and the scanner picks the highest-priority one.

Ambiguities Among Our 7 Token Types and Resolution:

Ambiguity	Input	Resolution
+ or - as sign vs. operator	+5, -3	Context: if preceded by operator/start, treat as sign (INT/FLOAT). Otherwise, treat as operator.
++ vs + then +	++	Longest match: DFA consumes both chars → INCDEC (priority 3 > single-char operator priority 10).
-- vs - then -	--	Same as above → INCDEC.
+digits vs +sign for INT/FLOAT	+42	The DFA reaches INT accepting state via B→D path. Scanner context decides if sign is part of literal.
true/false vs identifier	true	No conflict: booleans start lowercase, identifiers require uppercase first char. Mutually exclusive.
Integer vs Float prefix	3.14	Longest match: DFA reads 3 (INT), then sees '.' and continues to FLOAT. Backtracks to FLOAT.

## vs # + #	##x	Longest match: DFA consumes ## then all chars until newline → COMMENT (priority 2, highest).

Important Note on Signed Literals (+/-): In most real scanners, + and - are tokenized as separate operators, and the sign is resolved during parsing (e.g., unary minus). Our DFA supports both paths: from the start state, + leads to state B which can continue to digits (signed INT/FLOAT) or to another + (INCDEC). The scanner implementation in Task 1.2 will use context (previous token type) to decide which interpretation is correct.

Summary

Aspect	Value
Individual NFAs	7 (one per token type)
Total NFA states (individual)	3 + 13 + 2 + 3 + 10 + 2 + 5 = 38
Combined NFA states	39 (38 + 1 new start state S)
Combined NFA ε-transitions	7 (from S) + 3 (optional signs in INT, FLOAT, FLOAT-exp)
Combined DFA states (before min)	30 (including dead state Z)
Merged state pairs	J ≡ K (INCDEC), U ≡ AA (BOOL)
Minimized combined DFA states	28
Accepting states	13 (across 6 token types)

Key Design Decisions:

1. **Identifier length constraint** ({0,30} → max 31 chars) is enforced by a counter during scanning, not by expanding 31 DFA states. This keeps the combined DFA manageable.

2. **Float decimal precision** ({1,6} digits) IS encoded in the DFA via states F4-F9/P-AD, as this is essential for correctly rejecting floats with >6 decimal places.

3. **Pattern matching priority** is handled by the scanner logic, not the DFA. When multiple tokens could match (e.g., 'true' could be a keyword or boolean), the priority order from Section 3.12 of the assignment determines which token type wins.

4. **Longest match principle** is implemented by continuing DFA transitions as long as possible and backtracking to the last accepting state when no further transition exists.